Energy Consumption Prediction
using
Machine Learning and Deep Learning

Name : Kesigan Mukuntharathan

Assessment : Internship Assessment

Date : 12th April 2026

Link - Energy Consumption Prediction

Table of Contents

1.Introduction 4
2. Exploratory Data Analysis (EDA) 5
2.1 Dataset Overview 5
2.2 Summary Statistics 5
2.3 Time-Series Trend Analysis 6
2.4 Hourly Consumption Pattern 7
2.5 Daily Consumption Pattern 8
2.6 Distribution of Energy Consumption 9
2.7 Outlier Detection 10
2.8 Correlation Analysis 11
2.9 Rolling Trend Analysis 12
2.10 Autocorrelation Analysis 13
3. Data Preprocessing 14
3.1 Data Loading and Structure 14
3.2 Handling Duplicates 14
3.3 Handling Missing Values 14
3.4 Outlier Detection and Treatment 14
3.5 Time-Based Data Splitting 15
3.6 Feature Scaling 15
4. Feature Engineering 16
4.1 Time-Based Features 16
4.2 Lag Features 16
4.3 Rolling Window Features 17
4.4 Interaction Features 17
4.5 Domain-Specific Features 18
4.6 Cleaning After Feature Engineering 18
4.7 Feature Selection 18

4.8 Interpretation of Selected Features 19
4.9 Outcome of Feature Engineering 19
5. Model Design 20
5.1 Baseline Models 20
5.2 Deep Learning Model Architecture 20
5.3 Network Architecture 21
5.4 Activation Function 22
5.5 Optimizer 22
5.6 Loss Function 22
5.7 Regularization 22
5.8 Model Flexibility 23
5.9 Training Strategy 23
6. Results and Evaluation 24
6.1 Baseline Model Performance 24
6.2 Deep Learning Model Performance 25
6.3 Visual Evaluation 26
7. Model Optimization 31
7.1 Hyperparameter Optimization 31
7.2 Optimized Model Configurations 31
7.3 Optimized Model Performance 32
7.4 Comparison After Optimization 33
7.5 Final Model Selection 33
7.6 Discussion of Optimization Results 33
7.7 Visualization of Results 34
8. Challenges and Solutions 35
9. Conclusion 36

1.Introduction

Accurate energy consumption forecasting is essential for improving energy efficiency, reducing operational costs, and enabling intelligent energy management systems. With the increasing adoption of smart meters and IoT-based monitoring systems, large volumes of time-series data are being collected at frequent intervals. This data provides an opportunity to apply machine learning and deep learning techniques to understand consumption patterns and predict future energy usage.

This project focuses on building a predictive model for household appliance energy consumption using historical sensor data, environmental measurements, and time-related features. The dataset includes temperature, humidity, and weather-related variables collected at regular time intervals, making it suitable for time-series modeling.

The objective of this project is to:

- Analyze and preprocess real-world time-series energy consumption data
- Engineer meaningful features such as time-based indicators, lag features, and rolling statistics
- Develop baseline machine learning models to establish performance benchmarks
- Implement deep learning models (LSTM and GRU) to capture temporal dependencies
- Optimize model performance using hyperparameter tuning and regularization techniques
- Compare classical machine learning models with deep learning approaches to identify the most effective solution

By following a structured machine learning workflow - from data preprocessing and feature engineering to model optimization and evaluation - this project aims to demonstrate both practical machine learning skills and a strong understanding of time-series modeling techniques.

The results showed that traditional machine learning models outperformed deep learning models for this dataset due to strong feature engineering.

2. Exploratory Data Analysis (EDA)

The goal of exploratory data analysis is to understand the structure, patterns, and relationships within the dataset before performing feature engineering and modeling. The dataset contains time-series sensor readings collected at regular intervals, including indoor and outdoor temperature, humidity, weather conditions, and energy usage by appliances.

2.1 Dataset Overview

The dataset contains **19,735** records and **29** features, including:

- **Target Variable:** Appliances (energy consumption in Wh)
- **Environmental Features:**
  - ✓ Indoor temperatures (T1-T9)
  - ✓ Indoor humidity (RH_1-RH-9)
  - ✓ Outdoor weather (T_out, RH_out, Windspeed, Visibility, Tdewpoint, etc.)
- **Timestamp:** date (10-minute intervals)

The dataset is continuous and time-dependent, making it suitable for time-series modeling.

2.2 Summary Statistics

Descriptive statistics were used to understand the data distribution.
Key observations:

- Energy consumption (Appliances) is highly skewed, with values ranging from **10 Wh to over 1000 Wh**.
- Most sensor values (temperature and humidity) have relatively narrow ranges and low variance.
- No missing values were observed in the dataset.

This indicates that preprocessing should focus more on **outlier handling** and **scaling**, rather than missing-value imputation.

2.3 Time-Series Trend Analysis

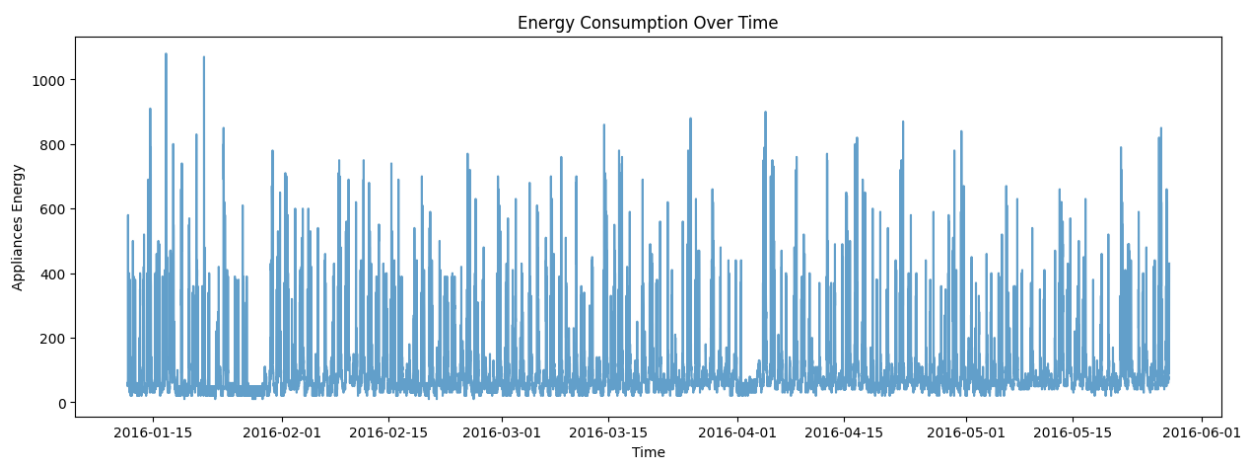
Energy consumption was visualized over time to identify temporal patterns and variability.

Observations:

- The time series exhibits high variability with frequent sharp spikes, indicating irregular appliance usage patterns.
- Energy consumption values are mostly concentrated at lower levels, with occasional extreme peaks exceeding 800–1000 Wh, confirming the presence of outliers.
- A repeating cyclical pattern is observed, suggesting strong daily seasonality in energy usage.
- The fluctuations appear consistent throughout the dataset duration, indicating stationary behavior in mean but not in variance.
- Periods of low consumption are followed by sudden spikes, implying that recent past values may influence future consumption.

Implications for Modeling:

- The presence of temporal dependencies justifies the use of sequence-based models such as LSTM and GRU.
- The observed spikes and irregular patterns support the need for:
  - ✓ Lag features
  - ✓ Rolling statistics
- The cyclical behavior motivates the inclusion of:
  - ✓ Hour-based features
  - ✓ Day-of-week indicators



2.4 Hourly Consumption Pattern

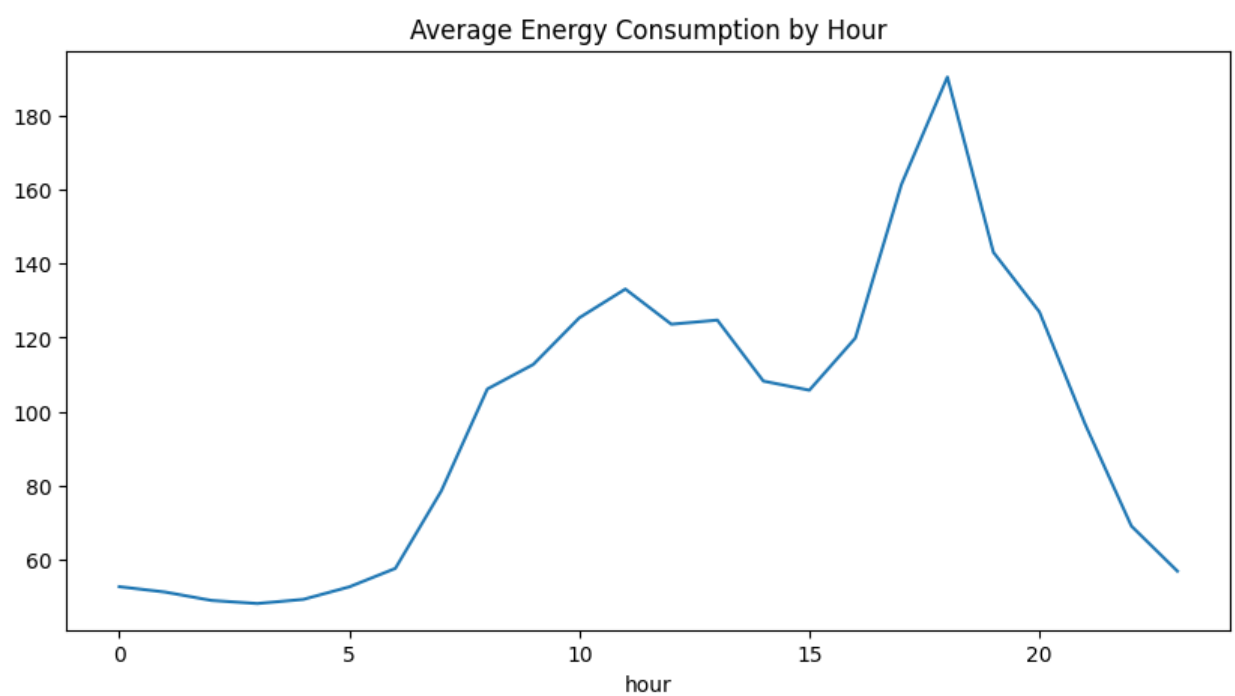
Average energy consumption was aggregated by hour of the day to analyze daily usage behavior.

Observations:

- Energy consumption follows a clear **daily cycle**, with low consumption during late night and early morning hours (00:00–05:00).
- Usage gradually increases after **06:00**, corresponding to morning household activity.
- Two distinct peaks are observed:
  - ✓ **Morning peak** around **09:00–11:00**
  - ✓ **Evening peak** around **17:00–19:00**, which represents the highest energy consumption period.
- After **20:00**, consumption gradually decreases, indicating reduced household activity during late evening hours.

Implications for Feature Engineering and Modeling:

- The observed daily pattern indicates strong hourly seasonality.
- This justifies the inclusion of:
  - ✓ hour as a feature
  - ✓ is_peak_hour indicators
  - ✓ Time-based lag features
- The predictable hourly structure supports the use of time-aware models such as LSTM and GRU.



2.5 Daily Consumption Pattern

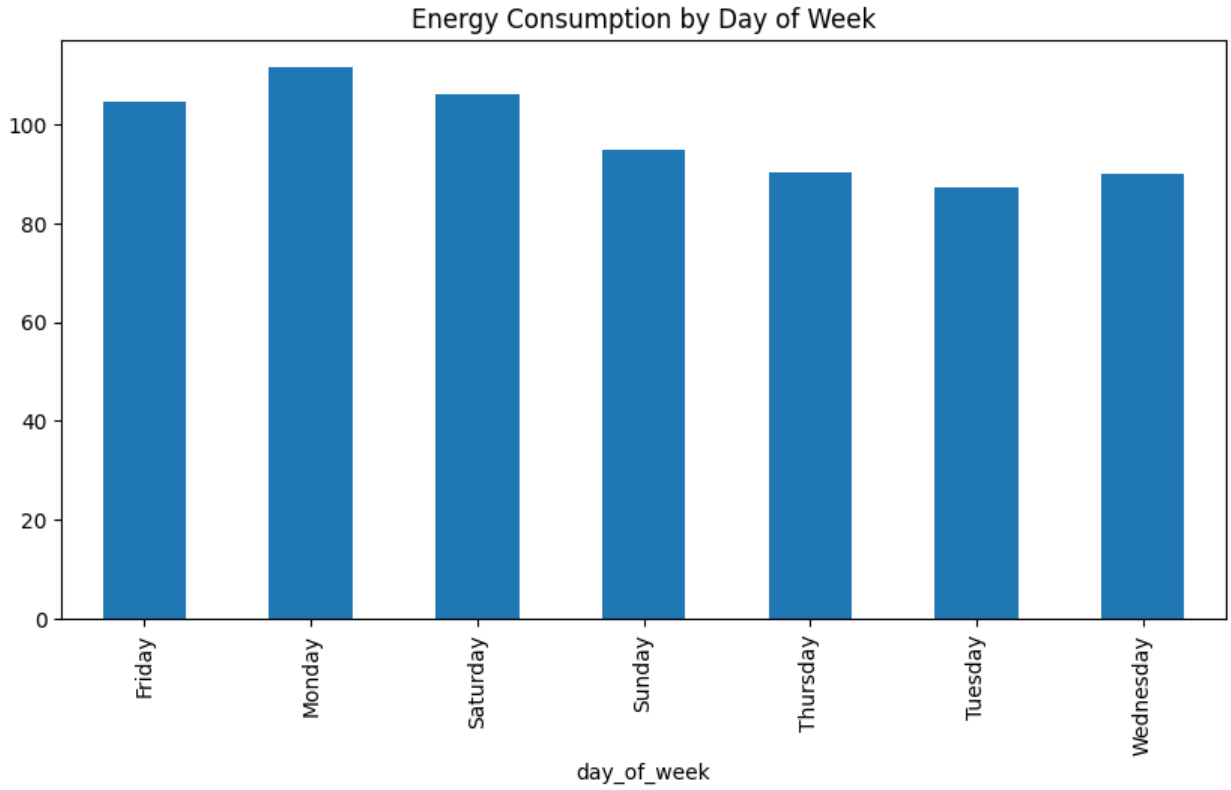
Energy consumption was aggregated by day of the week to analyze weekly usage behavior.

Observations:

- The average energy consumption varies moderately across days of the week.
- Monday shows the highest average energy consumption, while Tuesday and Thursday show the lowest usage.
- The difference between weekdays and weekends is present but not substantial, indicating that weekly seasonality is moderate rather than strong.
- Weekend days (Saturday and Sunday) show slightly lower consumption compared to peak weekday values, but the overall pattern remains consistent across the week.

Implications for Feature Engineering and Modeling:

- The presence of moderate weekly variation justifies the inclusion of:
  - ✓ day_of_week as a feature
  - ✓ is_weekend as a derived indicator
- However, since weekly variation is not highly pronounced, daily and hourly features are expected to have stronger predictive power than weekday indicators.



2.6 Distribution of Energy Consumption

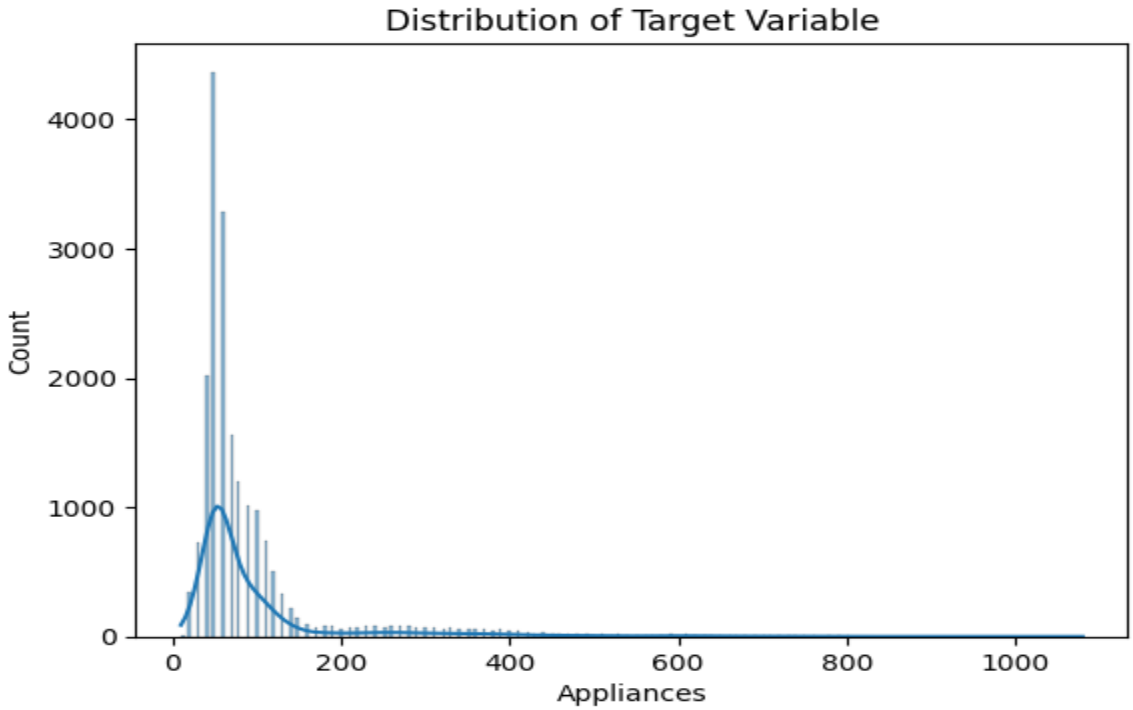
The distribution of the target variable (*Appliances*) was analyzed to understand its behavior and challenges for modeling.

Observations:

- The distribution is **strongly right-skewed**, with the majority of energy consumption values concentrated at lower ranges (below 150 Wh).
- A small number of observations represent high-energy consumption spikes, forming a long right tail extending beyond 1000 Wh.
- These high values are rare but important, as they represent periods of intensive appliance usage.

Implications for Modeling:

- The skewed distribution introduces an imbalance in regression, where models may bias toward predicting lower consumption values.
- High-consumption spikes are difficult to predict but are critical for accurate forecasting.
- This motivates:
  - ✓ Outlier detection and handling to reduce noise while preserving important peaks.
  - ✓ The use of Mean Squared Error (MSE), which penalizes large prediction errors and helps capture high-value spikes.
  - ✓ Careful evaluation using metrics such as MAE and RMSE to assess both average error and sensitivity to large errors.



2.7 Outlier Detection

A boxplot was used to analyze the presence of outliers in the target variable (*Appliances*).

Observations:

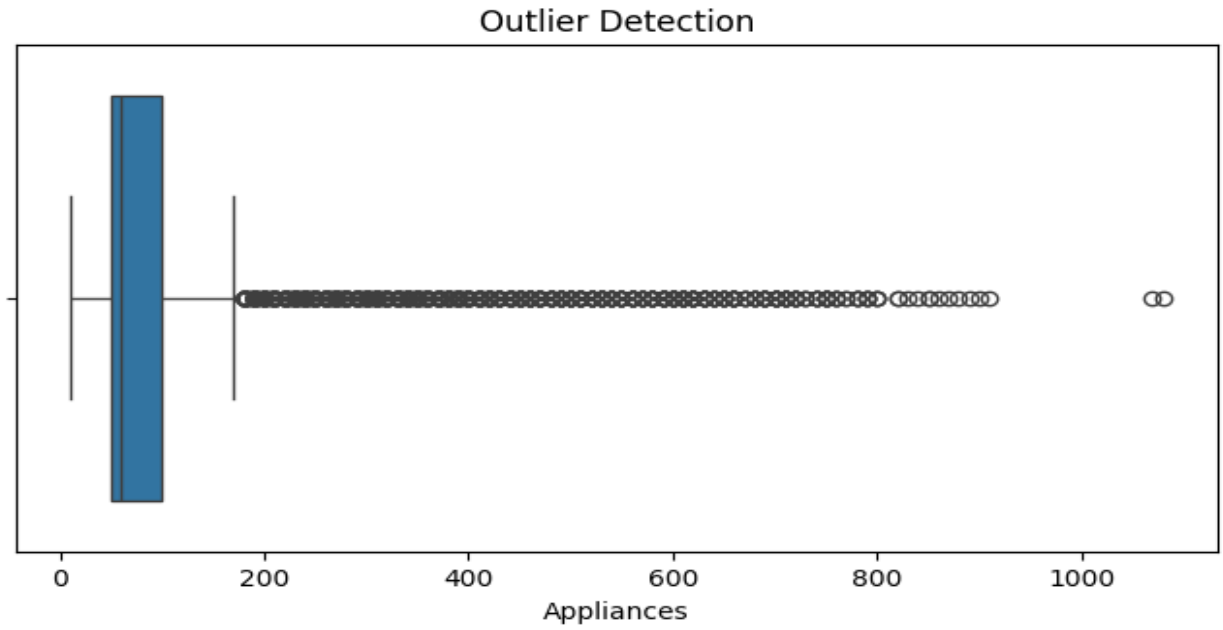
- The majority of energy consumption values fall within a relatively narrow interquartile range.
- A large number of data points appear beyond the upper whisker, indicating the presence of significant outliers.
- These outliers represent extreme energy consumption events and form a long tail in the data distribution.

Outlier Handling Approach:

- An **Interquartile Range (IQR)**-based method was applied to detect and remove extreme values.
- A total of **2,138** outliers were removed during preprocessing.
- This step reduces noise and prevents extreme values from disproportionately influencing the model during training.

Impact on Modeling:

- Removing extreme outliers improves model stability and prevents overfitting to rare spikes.
- It allows the model to better learn typical consumption patterns while still retaining meaningful variability.
- This preprocessing step supports more reliable performance of both tree-based and deep learning models.



2.8 Correlation Analysis

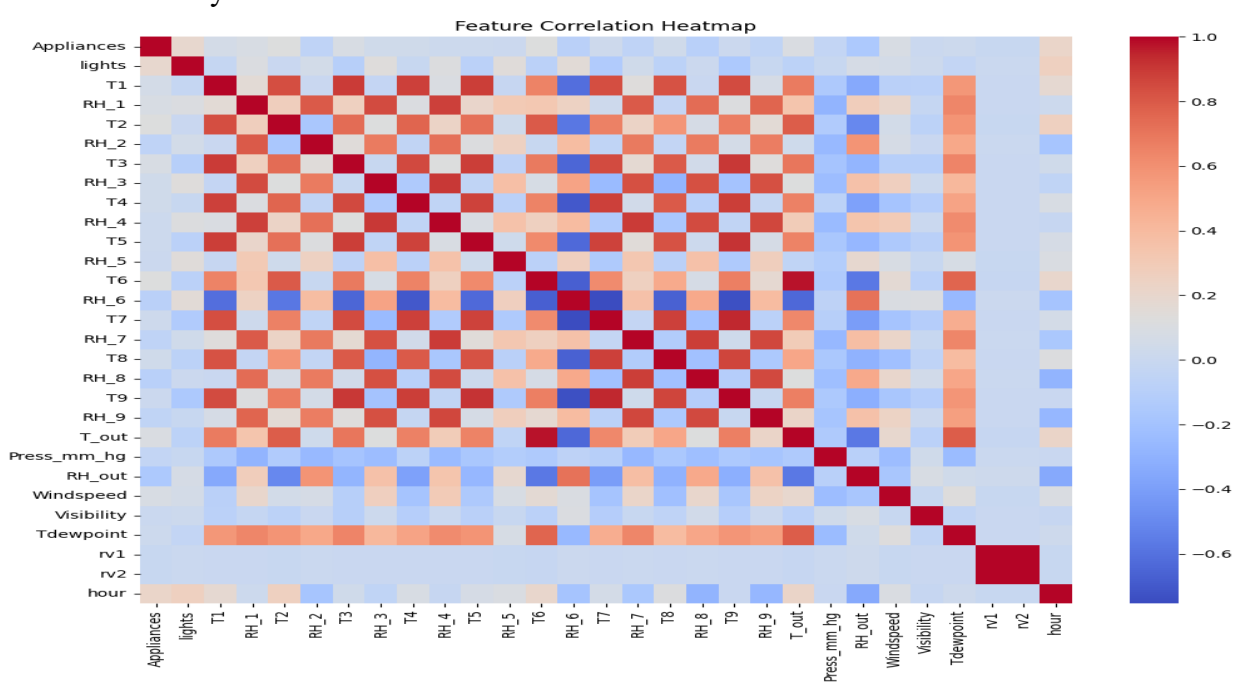
A correlation heatmap was used to analyze relationships between features and identify dependencies within the dataset.

Observations:

- Strong correlations are observed among temperature features (T1–T9) and humidity features (RH_1–RH_9), indicating significant multicollinearity among indoor sensor readings.
- The target variable (Appliances) shows weak to moderate correlation with most individual features, suggesting that no single feature alone can effectively predict energy consumption.
- Environmental variables such as outdoor temperature (T_out) and dew point (Tdewpoint) exhibit moderate relationships with energy usage.
- Features such as rv1 and rv2 show negligible correlation, indicating that they likely do not contribute meaningful information for prediction.

Implications for Modeling:

- The presence of multicollinearity justifies the use of:
  - ✓ Feature selection techniques (e.g., Random Forest importance)
  - ✓ Models that can handle correlated inputs effectively (e.g., tree-based models, neural networks)
- The weak correlation between individual features and the target highlights the need for:
  - ✓ Feature engineering (lag, rolling, interaction features)
  - ✓ Non-linear models to capture complex relationships
- Redundant or low-importance features (e.g., rv1, rv2) can be safely removed to improve model efficiency.



2.9 Rolling Trend Analysis

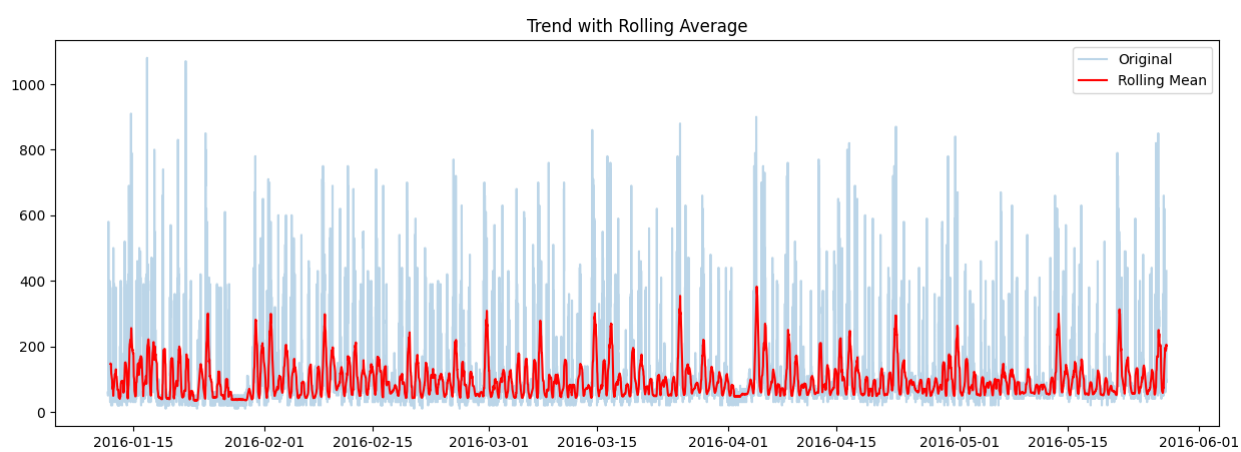
Rolling averages were computed to smooth short-term fluctuations and reveal underlying trends in energy consumption.

Observations:

- The original time series exhibits high variability with frequent spikes, indicating noisy behavior in raw energy consumption.
- The rolling average significantly reduces noise, highlighting a more stable trend over time.
- The smoothed curve reveals consistent patterns in baseline energy usage, which are not clearly visible in the raw data.
- While short-term spikes remain important, the rolling trend captures the overall consumption behavior more effectively.

Implications for Feature Engineering and Modeling:

- The presence of noise in the raw signal makes it difficult for models to learn meaningful patterns directly.
- Rolling features (e.g., rolling mean and rolling standard deviation) help:
  - ✓ Capture local trends
  - ✓ Reduce sensitivity to sudden spikes
  - ✓ Improve model stability
- This justifies the inclusion of:
  - ✓ rolling_mean_1h
  - ✓ rolling_mean_3h
  - ✓ rolling_std_1h
- These features enable models to learn both short-term variations and long-term trends, improving predictive performance.



2.10 Autocorrelation Analysis

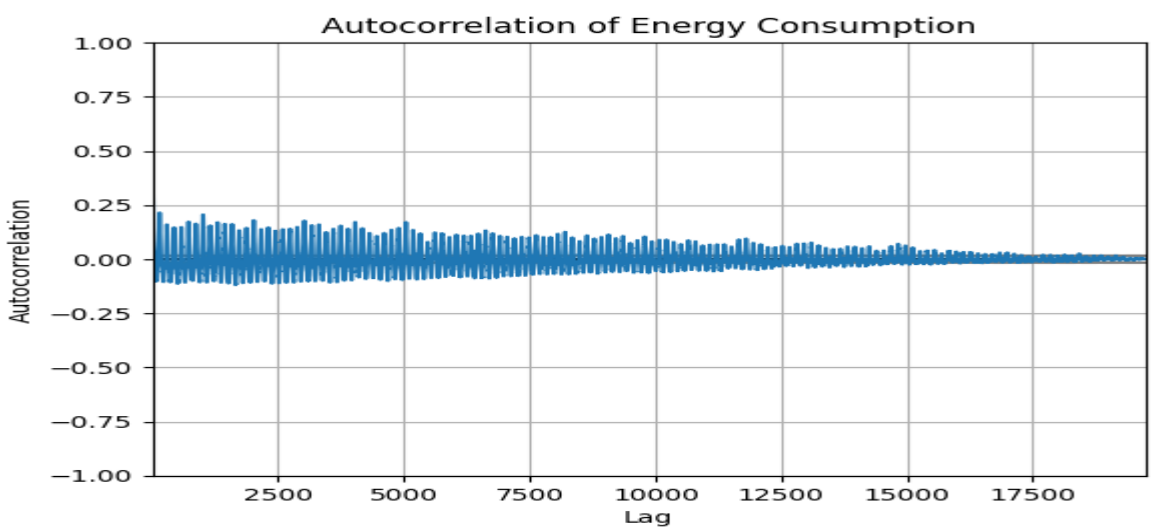
Autocorrelation analysis was conducted to evaluate the relationship between past and future energy consumption values.

Observations:

- The autocorrelation values are highest at small lags, indicating that recent energy consumption strongly influences future consumption.
- As the lag increases, the autocorrelation gradually decreases, showing that the influence of past values weakens over time.
- The presence of significant autocorrelation over multiple lags confirms that the dataset exhibits temporal dependency.

Implications for Feature Engineering and Modeling:

- The strong short-term autocorrelation justifies the inclusion of lag-based features (e.g., lag_1, lag_2, lag_3, lag_6).
- The persistence of autocorrelation over time supports the use of sequence-based deep learning models, such as:
    - ✓ Long Short-Term Memory (LSTM)
    - ✓ Gated Recurrent Unit (GRU)
- These models are capable of capturing temporal patterns and dependencies that traditional regression models cannot fully exploit.



3. Data Preprocessing

The dataset was prepared through a structured preprocessing pipeline to ensure data quality, reduce noise, and prevent data leakage before model training. The preprocessing steps included duplicate removal, missing value handling, outlier treatment, time-based splitting, and feature scaling.

3.1 Data Loading and Structure

The dataset contains **19,735 records** and **29 columns**, including:

- Timestamp (date)
- Target variable: Appliances
- Environmental and sensor variables such as temperature, humidity, pressure, visibility, and wind speed.

The dataset is a time-series dataset recorded at 10-minute intervals, making chronological ordering essential.

3.2 Handling Duplicates

Duplicate records were checked and removed to avoid data redundancy:

Duplicates removed: 0

No duplicate records were found in the dataset.

3.3 Handling Missing Values

Initial inspection of the dataset showed:

Missing values before: 0
Missing values after: 0

Although no missing values were present, a robust missing-value handling pipeline was implemented to ensure data consistency:

- The timestamp column was set as the index.
- **Time-based interpolation** was applied to preserve temporal continuity.
- Any remaining missing values were filled using **backward fill** and **forward fill**.

This approach is suitable for time-series data because it preserves temporal trends and avoids introducing artificial discontinuities.

Although no missing values were present, a robust handling pipeline was implemented to ensure reliability for real-world scenarios where missing sensor data may occur.

3.4 Outlier Detection and Treatment

Outliers in energy consumption can significantly distort model learning. Outliers were detected and removed using the **Interquartile Range (IQR)** method applied to the Appliances variable.
Outliers removed (IQR): 2138

This means that **2,138 extreme values** were removed from the dataset.

Justification:

- Energy consumption exhibited a right-skewed distribution with extreme spikes.
- Removing extreme outliers prevents the model from overfitting to rare abnormal events.
- IQR-based filtering is robust and does not assume normal distribution.

Z-score analysis was initially explored for outlier detection; however, IQR-based filtering was selected because it is more robust for skewed distributions and does not assume normality.

3.5 Time-Based Data Splitting

To prevent data leakage and respect temporal order, the dataset was split chronologically:
Train: (12313, 44)
Validation: (2639, 44)
Test: (2639, 44)
The dataset was split into three sets:

- **Training set (70%)**
- **Validation set (15%)**
- **Test set (15%)**

The validation set was used for hyperparameter tuning and early stopping, while the test set was used for final evaluation.
This mirrors real-world forecasting, where future data must not influence past training.

3.6 Feature Scaling

Neural networks and distance-based algorithms are sensitive to feature scales. Therefore, Min-Max Scaling was applied to the input features.

Scaling was performed using MinMaxScaler, where feature values were transformed to a range between 0 and 1.

- The scaler was fitted only on the training data and then applied to the validation and test sets to avoid data leakage.
- Only input features were scaled, while the target variable (Appliances) remained in its original scale to preserve interpretability.
- The timestamp column (date) was excluded from scaling, as it is used only for temporal ordering.

After preprocessing and feature engineering, the dataset contained 44 columns, including engineered features.
Dataset Split:

- Training set: 70% Validation set: 15% Test set: 15%

Reason for choosing Min-Max Scaling:

- Keeps feature values bounded within a fixed range.
- Improves convergence for gradient-based optimization in deep learning models.
- Suitable for models sensitive to feature magnitude differences.

4. Feature Engineering

Feature engineering plays a critical role in improving model performance, especially for time-series forecasting tasks. In this project, several categories of features were created to capture temporal patterns, historical dependencies, and domain-specific behavior in energy consumption.

4.1 Time-Based Features

To capture periodic and seasonal patterns in energy usage, the following time-based features were extracted from the timestamp:

- **Hour of the day (hour)**
- **Day of the week (day_of_week)**
- **Month (month)**
- **Weekend indicator (is_weekend)**

These features allow the model to learn:

- Daily consumption cycles
- Weekday vs weekend behavior
- Seasonal trends across months

Why this is important:
Energy consumption often varies by time of day (e.g., peak hours) and by day type (weekday vs weekend).

Although the month feature was created to capture long-term seasonality, it was not selected by the feature selection process, indicating that short-term temporal patterns dominate energy consumption.

4.2 Lag Features

Lag features were created to capture historical energy consumption patterns:

- lag_1
- lag_2
- lag_3
- lag_6

Each lag represents the energy consumption from previous time steps.

Why this is important:
Energy consumption is strongly autocorrelated, meaning that recent past consumption is one of the strongest predictors of future usage.

Since the data is recorded at 10-minute intervals, lag_1 represents 10 minutes, lag_2 represents 20 minutes, lag_3 represents 30 minutes, and lag_6 represents 1 hour of past energy consumption.

4.3 Rolling Window Features

Rolling statistics were computed to capture short-term trends and variability:

- **Rolling mean (1 hour)** – rolling_mean_1h
- **Rolling mean (3 hours)** – rolling_mean_3h
- **Rolling standard deviation (1 hour)** – rolling_std_1h

These features help the model learn:

- Short-term trends
- Smooth consumption patterns
- Sudden fluctuations

Why this is important:
Rolling features reduce noise and highlight underlying patterns in energy usage.

Since the dataset is sampled at 10-minute intervals, a 1-hour rolling window corresponds to 6 time steps, and a 3-hour window corresponds to 18 time steps.

This aligns with the rolling trend analysis observed during EDA, where smoothed curves revealed clear daily consumption cycles.

4.4 Interaction Features

To model relationships between environmental factors, interaction features were created:

- temp_humidity = T_out * RH_out
- indoor_temp_humidity = T1 * RH_1

These interactions allow the model to learn combined effects of:

- Temperature and humidity
- Indoor comfort conditions

Why this is important:
Energy usage (e.g., heating or cooling) often depends on combined temperature and humidity rather than each variable individually.
Although interaction features were created to capture combined environmental effects, they were not selected during feature selection, suggesting that direct temporal features were more informative than combined environmental interactions.

4.5 Domain-Specific Features

Domain knowledge was used to introduce meaningful behavioral features:

- **Peak hour indicator (is_peak_hour)**
  Identifies evening peak hours (17:00–20:00) when energy demand typically rises.
- **Night indicator (is_night)**
  Captures low-activity periods during nighttime hours (00:00–05:00).

Why this is important:
These features help the model explicitly learn typical human activity patterns influencing energy consumption.
These features were motivated by EDA results, which showed higher energy consumption during evening hours and lower usage during nighttime.

4.6 Cleaning After Feature Engineering

Due to lag and rolling operations, missing values were introduced at the beginning of the dataset. These rows were removed:

Dropped NaN after feature engineering

This ensures all samples used for training contain complete feature information.

4.7 Feature Selection

To avoid overfitting and reduce model complexity, feature selection was performed using a **Random Forest Regressor** trained only on the training data.

Features were ranked by importance, and only features above a threshold were selected.

Top Selected Features

[rolling_mean_1h, lag_1, rolling_std_1h, lag_3, hour, rolling_mean_3h, lag_2, lag_6, Press_mm_hg, T8,RH_5]

Feature selection was performed using only the training dataset to prevent data leakage and ensure unbiased evaluation.
Random Forest was selected for feature importance estimation because it can capture non-linear relationships and interactions between features without requiring feature scaling.
A feature importance threshold of 0.006 was selected to retain informative features while removing weak predictors, balancing model complexity and performance.

Feature selection was performed strictly on the training dataset to avoid data leakage.

4.8 Interpretation of Selected Features

- **Rolling and lag features** dominate importance, confirming strong temporal dependency.
- **Hour of day** captures daily usage cycles.
- **Environmental features** like pressure and temperature still contribute, but less strongly.

Key Insight:

Recent energy consumption history is the strongest predictor of future usage, highlighting the importance of temporal dependencies in time-series forecasting.

4.9 Outcome of Feature Engineering

- Improved feature representation of temporal and behavioral patterns
- Reduced dimensionality through feature selection
- Improved model stability and generalization

Overall, feature engineering significantly improved the model's ability to capture temporal dynamics and reduced reliance on raw sensor data alone.

5. Model Design

The objective of the modeling stage is to predict future energy consumption using both baseline machine learning models and deep learning models. Two types of models were implemented:

- **Baseline Models** (for performance benchmarking)
- **Deep Learning Models** (for time-series learning)

5.1 Baseline Models

Two traditional machine learning models were implemented to establish benchmark performance:

Linear Regression

- Serves as a simple baseline model.
- Assumes a linear relationship between input features and energy consumption.
- Helps evaluate whether more complex models are necessary.

Random Forest Regressor

- Ensemble learning model based on decision trees.
- Captures non-linear relationships and feature interactions.
- Provides feature importance values used later in feature selection.

Purpose of baseline models:
These models act as reference points to evaluate whether deep learning provides meaningful improvement.

5.2 Deep Learning Model Architecture

A recurrent neural network (RNN)-based architecture was implemented to capture temporal dependencies in energy consumption.

The model supports two architectures:

- **Long Short-Term Memory (LSTM)**
- **Gated Recurrent Unit (GRU)**

Both architectures are suitable for time-series forecasting because they retain information over time.

5.3 Network Architecture

The deep learning model consists of:

- **Input Layer:**
  Accepts sequences of time-series features.
  Each input consists of a sequence of past time steps, enabling the model to learn temporal dependencies.
- **Recurrent Layer (LSTM / GRU):**
  - ✓ Number of layers: 2
  - ✓ Hidden units: Configurable (default: 64)
  - ✓ Captures temporal patterns and dependencies.
- **Fully Connected Layers:**
  The linear layer transforms hidden states into a lower-dimensional representation.
  - ✓ ReLU activation
  - ✓ Dropout layer for regularization
  - ✓ Final linear layer outputs a single predicted value.

Architecture Flow:

Input Sequence
↓
LSTM / GRU Layer(s)
↓
Fully Connected Layer
↓
ReLU Activation
↓
Dropout
↓
Output Layer (Prediction)

The model receives sequential input data, where each input consists of a sequence of past time steps. This allows the recurrent layers to learn temporal dependencies and patterns over time.

The model uses only the final time-step output from the recurrent layer, allowing the network to summarize the entire input sequence into a single representation for prediction.

The model was trained using both training and validation sets, where validation loss was used to monitor performance and guide early stopping.

5.4 Activation Function

ReLU activation is applied after the first fully connected layer to introduce non-linearity.

Why ReLU?

- Prevents vanishing gradient problems.
- Allows faster convergence.
- Introduces non-linearity, improving model capacity.

5.5 Optimizer

The model uses the **Adam optimizer**.

Why Adam?

- Adaptive learning rate optimization.
- Faster convergence compared to standard gradient descent.
- Effective for non-stationary and noisy data.

The optimizer and loss function are defined during the training phase in the training pipeline.

5.6 Loss Function

The loss function used is:

- **Mean Squared Error (MSE)**

Why MSE?

- Standard loss function for regression problems.
- Penalizes large errors more heavily.
- Suitable for continuous value prediction such as energy consumption.

5.7 Regularization

To reduce overfitting:

- Dropout is applied after the activation layer to reduce overfitting by randomly deactivating neurons during training.
- Dropout randomly disables neurons during training, preventing co-adaptation.

This improves generalization and reduces overfitting.

Early stopping was applied during training to prevent overfitting by monitoring validation loss and stopping training when no improvement was observed.

5.8 Model Flexibility

The model is designed to be flexible:

- Supports **LSTM** and **GRU**
- Allows tuning of:
  - ✓ Number of hidden units
  - ✓ Dropout rate
  - ✓ Number of layers
  - ✓ Model type (LSTM or GRU)

This flexibility enables experimentation and optimization.

5.9 Training Strategy

The training process included:
1.Train/Validation/Test split
2.Early stopping based on validation loss (patience = 7)
3.Grid search hyperparameter tuning
4.Model selection based on lowest validation loss

This approach ensures robust model evaluation and prevents overfitting.

6. Results and Evaluation

This section presents the performance of baseline models, deep learning models, and the optimized model. Evaluation was conducted using:

- **MAE (Mean Absolute Error)**
- **RMSE (Root Mean Squared Error)**
- **R^2 Score (Coefficient of Determination)**
- **MAPE (Mean Absolute Percentage Error)**

The validation set was used for hyperparameter tuning and early stopping, while the test set was reserved for final evaluation.

6.1 Baseline Model Performance

Two classical machine learning models were trained to establish a baseline.

Linear Regression

Metric	Value
MAE	8.3950
RMSE	12.1617
R^2	0.7844
MAPE	11.89%

Random Forest

Metric	Value
MAE	8.0347
RMSE	11.8069
R^2	0.7968
MAPE	11.35%

Observation:
Random Forest achieved the best baseline performance, indicating strong nonlinear relationships in the data.

6.2 Deep Learning Model Performance

Two deep learning architectures were trained:

- LSTM (Long Short-Term Memory)
- GRU (Gated Recurrent Unit)

A sequence length of 24 corresponds to approximately 4 hours of historical data (24 × 10-minute intervals).

LSTM Performance

Metric	Value
MAE	10.4871
RMSE	15.3391
R^2	0.6578
MAPE	14.63%

GRU Performance

Metric	Value
MAE	10.3030
RMSE	15.3196
R^2	0.6587
MAPE	14.25%

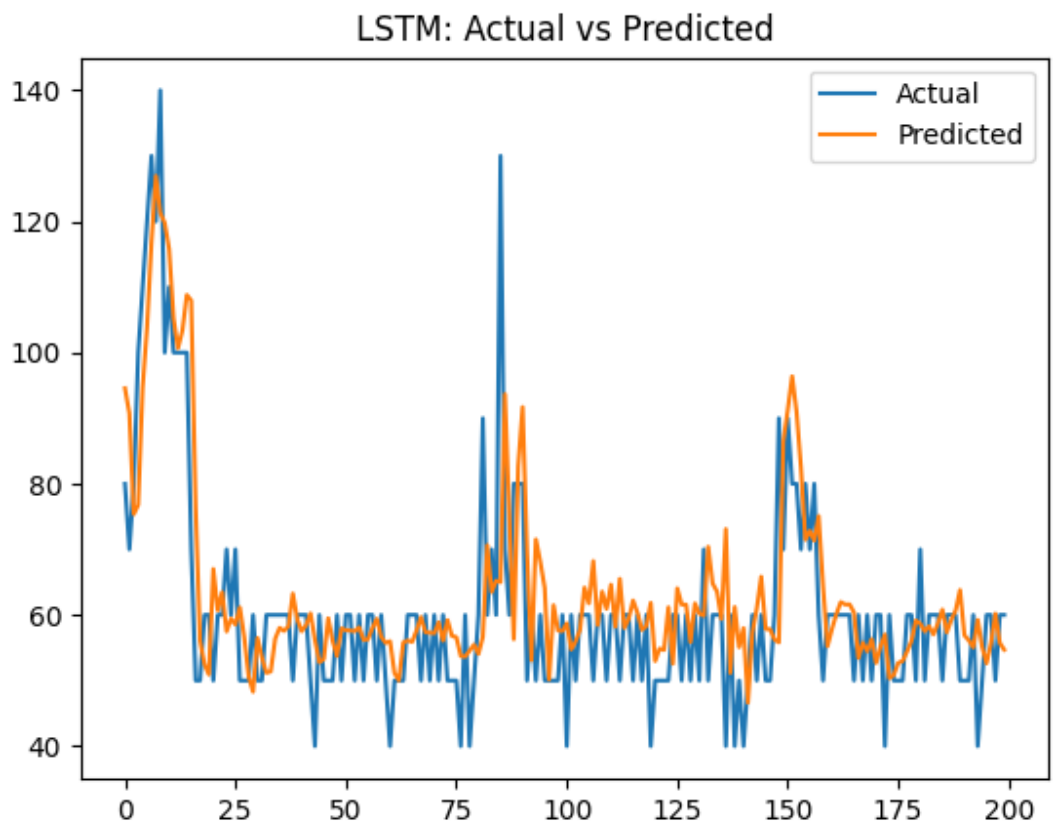
Observation:

- Deep learning models showed improvement after sufficient training but still underperformed compared to Random Forest.

6.3 Visual Evaluation

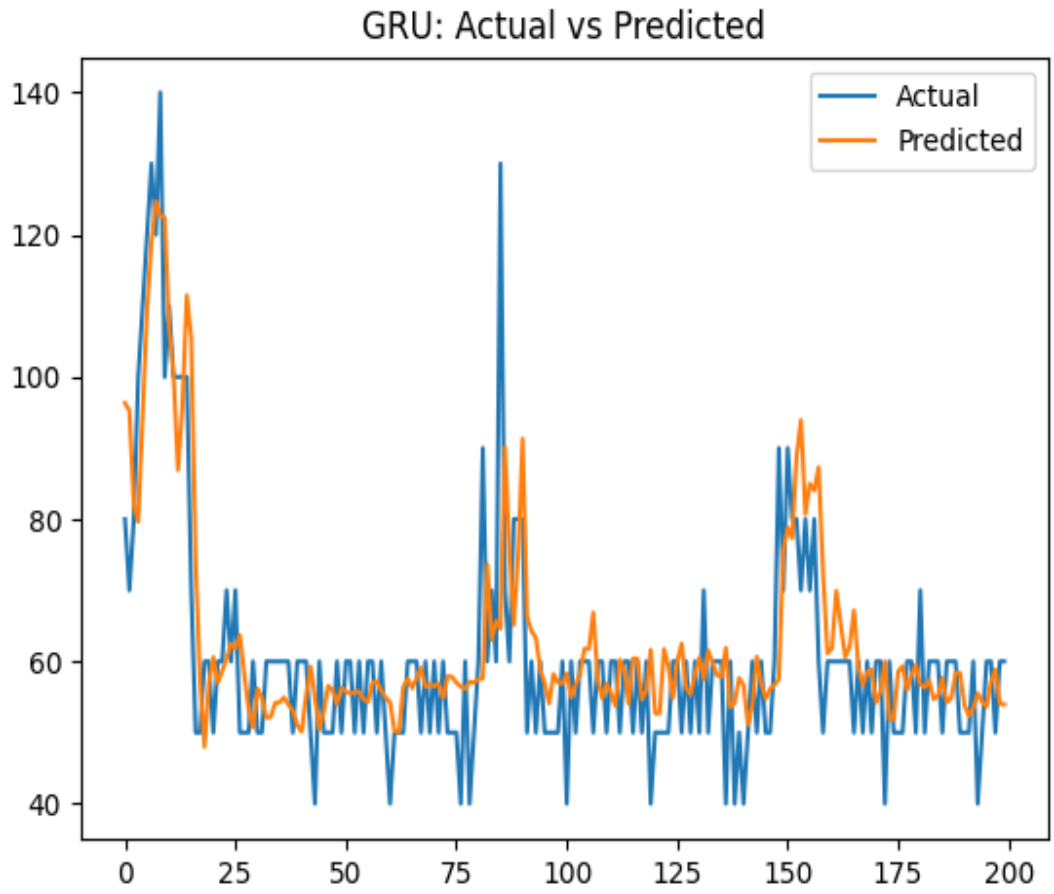
The following plots were generated for both LSTM and GRU models:

- Actual vs Predicted values



The Actual vs Predicted plot for the LSTM model shows that the model is able to follow the general trend of energy consumption. The predicted curve follows the overall shape of the actual values, indicating that the model successfully captures short-term fluctuations.

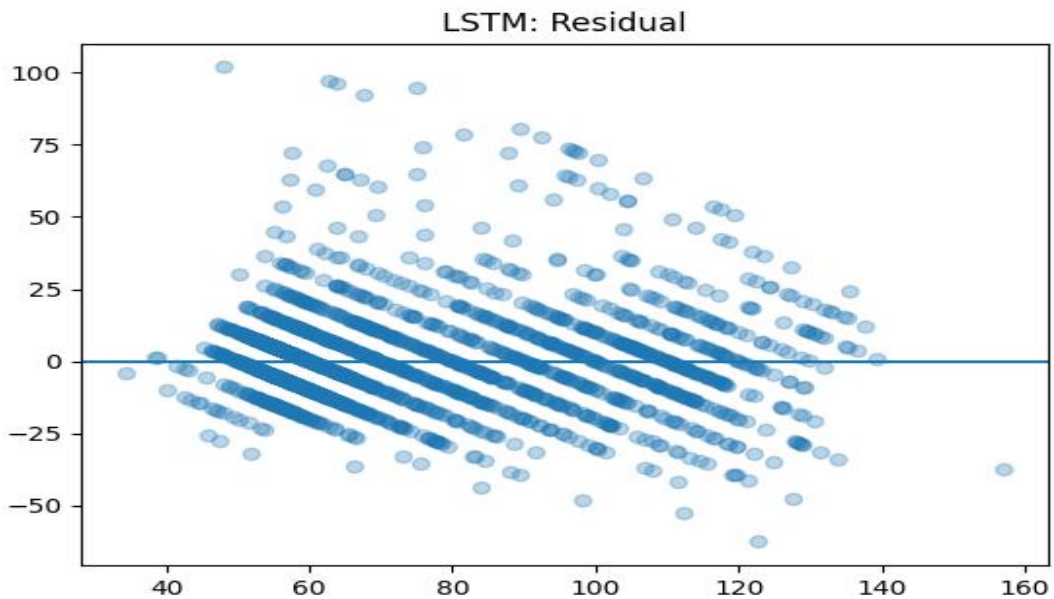
However, during sharp peaks in energy consumption, the model underestimates the true values. This suggests that while the LSTM captures overall temporal patterns, it struggles to model extreme spikes in energy usage.



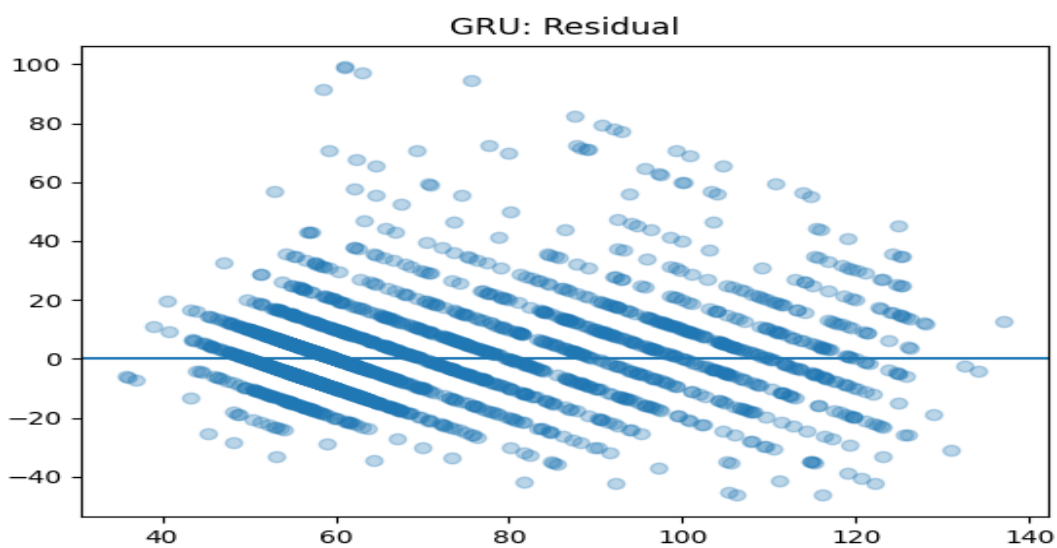
The GRU Actual vs Predicted plot shows behavior similar to the LSTM model. The predicted values closely follow the general consumption pattern but fail to fully capture extreme peak values.

Compared to LSTM, the GRU predictions appear slightly smoother, indicating better stability but also showing underestimation during high-consumption periods.

- Residual plot (error distribution)



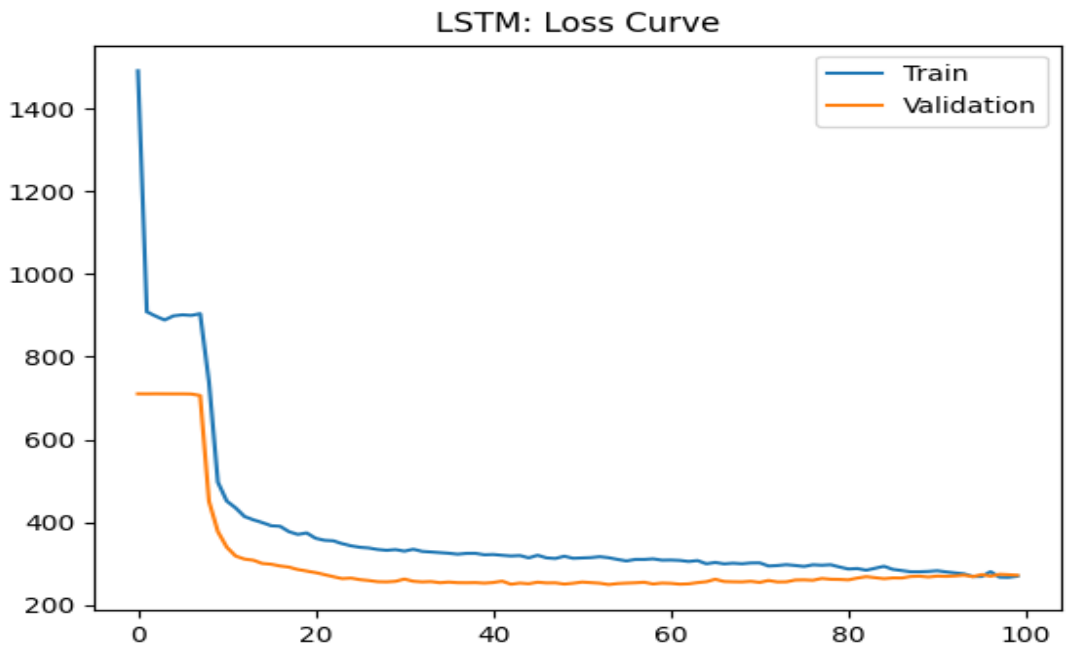
The residual plot for LSTM shows the distribution of prediction errors. Residuals are centered around zero, indicating no strong bias in the predictions.
However, the spread of residuals increases at higher predicted values, showing that the model struggles with large energy spikes. This suggests that errors increase during peak consumption periods.



The residual plot for GRU shows a similar pattern to LSTM, with most residuals clustered around zero. The spread of errors increases as predicted values increase, indicating larger errors during peak periods.

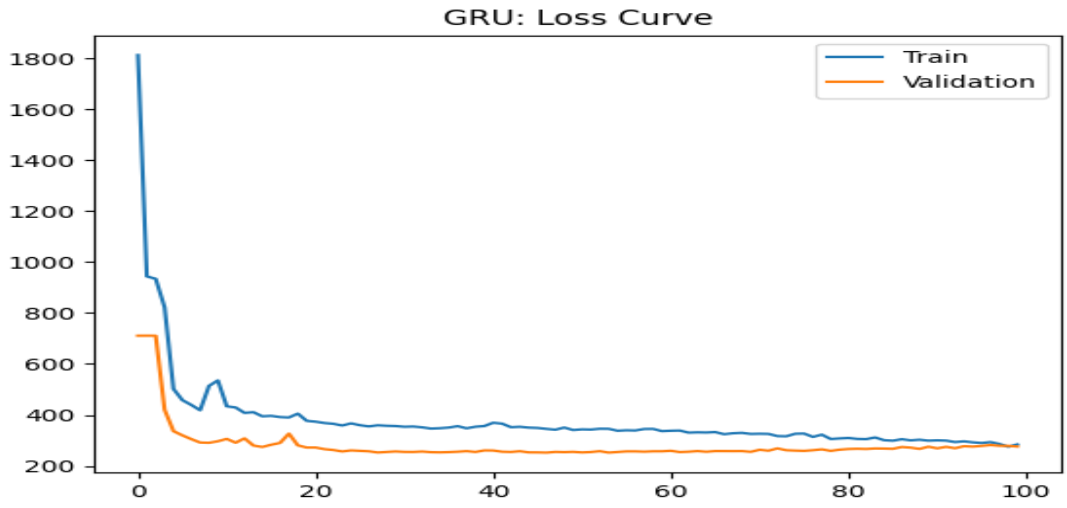
This confirms that both deep learning models face difficulty accurately predicting extreme energy spikes.

- Training vs Validation loss curves

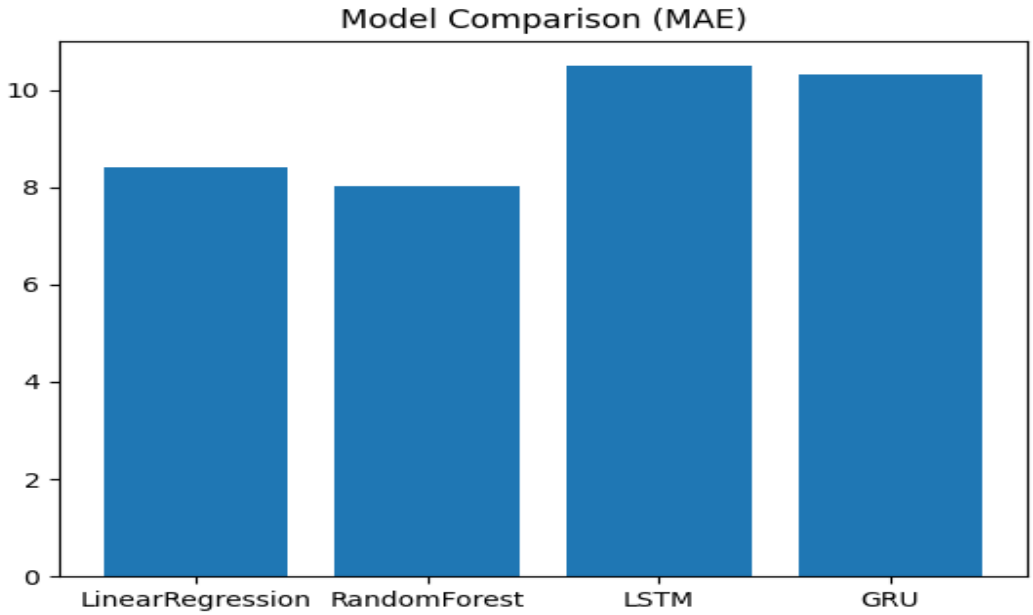


The training and validation loss curves show that the LSTM model converges smoothly over epochs. The training loss decreases steadily, while the validation loss stabilizes after several epochs.

This indicates that the model successfully learns from the data. However, the gap between training and validation loss suggests mild overfitting.



The GRU loss curves show faster convergence compared to LSTM. Both training and validation losses decrease rapidly and stabilize early.
The validation loss remains close to the training loss, indicating good generalization and less overfitting compared to LSTM.



The MAE comparison plot shows that the Random Forest model achieves the lowest error among all models. Both LSTM and GRU models show higher MAE, indicating that traditional machine learning models perform better on this dataset.

7. Model Optimization

To improve the performance of deep learning models, systematic hyperparameter tuning and regularization techniques were applied. The goal was to identify the best-performing configuration for both LSTM and GRU architectures.

7.1 Hyperparameter Optimization

- Optimization Techniques Used

1. Grid Search

- All possible combinations of hyperparameters were systematically evaluated.
- Each configuration was trained independently.
- The best configuration was selected based on lowest validation loss (MSE).

2. Early Stopping

- Training was monitored using validation loss.
- If validation loss did not improve for 7 consecutive epochs (patience = 7), training stopped early.
- This prevents:
  - ✓ Overfitting
  - ✓ Unnecessary computation

3. Model Selection Criterion

- Best model chosen based on:
  - ✓ Lowest validation loss during training
  - ✓ Final evaluation using RMSE and R^2

Hyperparameters optimized included learning rate, hidden layer size, and dropout rate. Optimization targeted minimization of validation loss (MSE).

7.2 Optimized Model Configurations

The best configurations identified for each model were:

Optimized LSTM

- Learning rate: **0.001**
- Hidden dimension: **64**
- Dropout: **0.2**
- Achieved lowest validation loss among LSTM variants

Optimized GRU

- Learning rate: **0.001**
- Hidden dimension: **64**
- Dropout: **0.2**

7.3 Optimized Model Performance

❖ **Optimized LSTM Performance**

Metric	Value
MAE	9.9943
RMSE	14.7535
R^2	0.6835
MAPE	14.24%

❖ **Optimized GRU Performance**

Metric	Value
MAE	10.1845
RMSE	14.8782
R^2	0.6781
MAPE	14.64%

7.4 Comparison After Optimization

7.4 Comparison After Optimization

Model	MAE	RMSE	R^2
Linear Regression	8.3950	12.1617	0.7844
Random Forest	8.0347	11.8069	0.7968
LSTM (Base)	10.4871	15.3391	0.6578
GRU (Base)	10.3030	15.3196	0.6587
Optimized LSTM	9.9943	14.7535	0.6835
Optimized GRU	10.1845	14.8782	0.6781

7.5 Final Model Selection

Between the optimized models:

- ✓ The optimized LSTM achieved the lowest RMSE and highest R^2 score.
- ✓ Therefore, the final selected model is: **LSTM** (Optimized)

7.6 Discussion of Optimization Results

- Hyperparameter tuning significantly improved both LSTM and GRU performance.
- Performance improvements observed:
  - ✓ Reduced RMSE
  - ✓ Increased R^2
- Optimization helped stabilize training and reduce overfitting.

However:

- Random Forest still outperformed deep learning models.
- This suggests:
  - ✓ The dataset is **tabular and feature-driven**
  - ✓ Engineered features (lags, rolling stats) already capture temporal patterns
  - ✓ Tree-based models are better suited for such structured data

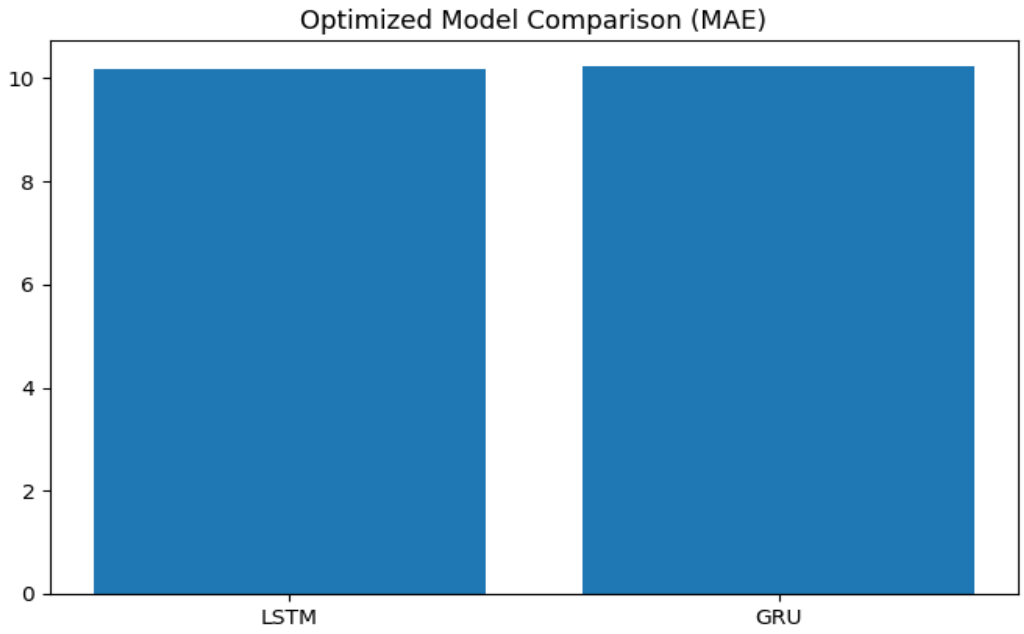
Key Insight:

Despite optimization, deep learning models underperformed compared to Random Forest because engineered lag and rolling features already captured most temporal dependencies, reducing the advantage of sequence models.

The results indicate that the dataset is highly feature-driven. The use of lag and rolling features already captures temporal dependencies effectively, reducing the advantage of sequence-based models such as LSTM and GRU. As a result, tree-based models like Random Forest outperform deep learning models.

7.7 Visualization of Results

Model Comparison Bar Chart



The optimized model comparison shows that hyperparameter tuning improves the performance of both LSTM and GRU models. Among the two, the optimized LSTM achieves a lower MAE compared to the optimized GRU, indicating slightly better prediction accuracy.

Despite these improvements, both deep learning models still underperform compared to the Random Forest model. This suggests that the engineered features, such as lag and rolling statistics, already capture most of the temporal dependencies in the dataset. As a result, traditional machine learning models are more effective for this problem than sequence-based deep learning models.

8. Challenges and Solutions

During the implementation of this project, several technical and modeling challenges were encountered. These challenges were addressed through systematic experimentation and design decisions.

8.1 Data Characteristics and Skewed Distribution

Challenge:
The target variable (energy consumption) exhibited a highly skewed distribution with extreme spikes. These spikes created difficulty for models to accurately predict peak consumption values.

Solution:
Outlier analysis was conducted using both Z-score and Interquartile Range (IQR) methods. IQR-based filtering was selected because it is robust to skewed distributions and does not assume normality. This helped reduce noise while preserving meaningful patterns.

8.2 Temporal Dependencies and Feature Representation

Challenge:
Energy consumption depends strongly on temporal behavior, making simple models insufficient for capturing sequential dependencies.

Solution:
Time-based features, lag features, and rolling window statistics were engineered to explicitly represent temporal patterns. This improved the model's ability to capture short-term and long-term dependencies.

8.3 Model Performance Limitations

Challenge:
Deep learning models (LSTM and GRU) underperformed compared to the Random Forest baseline.

Solution:
Hyperparameter optimization was performed using grid search and early stopping. This improved the performance of deep learning models but did not surpass the Random Forest model. This highlighted that the dataset is feature-driven rather than sequence-dominant.

9. Conclusion

Although deep learning models such as LSTM and GRU were able to capture temporal patterns, they did not outperform Random Forest. This is because the engineered features already captured most of the temporal dependencies, making the dataset more suitable for tree-based models.

The optimized LSTM model was selected as the best deep learning model; however, Random Forest remains the overall best-performing model.

Future Work

Potential improvements and extensions include:

- Using attention-based models or Transformer architectures for better long-range dependency modeling.
- Applying hybrid models that combine tree-based models with neural networks.
- Incorporating external data such as weather forecasts or occupancy patterns to improve prediction accuracy.

References

- Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.
- Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation.